USER GUIDE

turbofit

Turbospectrum synthesis, FERRE grids, NLTE
and line-by-line abundances with CNO model atmospheres

Version 0.1 · November 2026

Synthesis · NLTE · LSFs · FERRE · radial velocities · automatic line masks · line-by-line abundances

Contents

Overview

1. Installation

- 1.1 Turbospectrum v20
- 1.2 Python packages
- 1.3 Model atmospheres
- 1.4 NLTE data (only for NLTE mode)
- 1.5 FERRE (only for stellar parameters)
- 1.6 Quick check

2. Synthesising spectra

- 2.1 One spectrum: `synthesis.synthesize()`
- 2.2 Many abundance patterns for one star: `abund.py`
- 2.3 Typical run times

3. NLTE

- 3.1 When to use it
- 3.2 Setting it up
- 3.3 One NLTE spectrum
- 3.4 What happens under the hood
- 3.5 NLTE in grids and line fits

4. Broadening, LSFs and resampling

- 4.1 The one call you need: `broaden.convolve()`
- 4.2 Describing your instrument
- 4.3 Choosing the output grid
- 4.4 Doppler shifts
- 4.5 How accurate is it?

5. Building grids for FERRE

- 5.1 The short version
- 5.2 Broadening on the fly with any LSF: `broaden_kw`
- 5.3 NLTE grids
- 5.4 FERRE grids on log-λ pixels
- 5.5 Axis order (verified)

6. Radial velocities

- 6.1 Cross-correlation, then χ^2
- 6.2 How well calibrated are the errors?
- 6.3 Per-line RV offsets

7. Stellar parameters with FERRE

- 7.1 Fitting
- 7.2 Controlling FERRE
- 7.3 From FERRE to line-by-line fitting
- 7.4 The lower-level pieces
- 7.5 Three FERRE behaviours to know about

8. Line-by-line abundances

- 8.1 Setting up a `LineFitter`
- 8.2 The two modes
- 8.3 Line masks
- 8.4 Fitting
- 8.5 Macroturbulence: `vmac=`
- 8.6 Measurement errors on the abundances
- 8.7 Systematic errors from the stellar parameters
- 8.8 Results
- 8.9 Plots
- 8.10 How well does it work?

9. The full pipeline

10. Troubleshooting and known quirks

- 10.1 Errors you may see
- 10.2 Lines flagged in `linefit`
- 10.3 Known quirks (handled for you)
- 10.4 Getting help

11. API reference

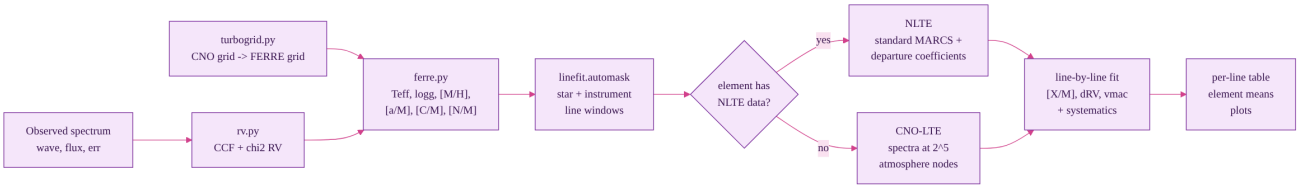
Overview

turbofit is a Python toolkit around **Turbospectrum v20**. It covers:

- synthesising spectra, one at a time or many abundance patterns per star;
- building FERRE grids;
- measuring radial velocities;
- fitting stellar parameters with FERRE;
- fitting abundances line by line.

For each element you choose how it is synthesised:

- **CNO-LTE**: APOGEE MARCS atmospheres with [C/M] and [α/M] nodes, in LTE.
- **NLTE**: standard MARCS atmospheres plus departure coefficients, for elements that have NLTE data.



Conventions used throughout

- **Abundances.** `mh` is [M/H], `am` is [α/M], `cm` is [C/M] and `nm` is [N/M].
  - Individual elements are given as `[X/M]` with `elems=[['Mg', 0.3], ['Al', -0.1]]`.
  - The α elements are O, Ne, Mg, Si, S, Ar, Ca and Ti.
  - Solar abundances are Grevesse et al. (2007), which is also Turbospectrum's default.
- **Wavelengths** are in Å. They follow your line lists, so air or vacuum depending on what you use.
- **Spectra passed to the fitting routines** must be continuum-normalised, with 1σ errors per pixel.
  - Bad pixels can be marked with `err = inf` or `nan`.
- **Radial velocities** are in km/s, with positive meaning receding. Observed spectra stay in the observed frame; turbofit applies the shift.
- **Imports are flat.** Put the turbofit folder on your `PYTHONPATH` (or `sys.path`) and use `import synthesis, linefit, ...`.

[X/M] or [X/Fe]?

turbofit works in **[X/M]** throughout: in the inputs (`elems`, `am`, `cm`, `nm`) and in the line-by-line results (`xm`). This follows the APOGEE/ASPCAP convention, where [M/H] (FERRE's METALS) is the overall scaling and each element is quoted relative to it.

- `mh` is added to *every* element heavier than He, **Fe included**. So by default [Fe/H] = [M/H] and **[X/M] = [X/Fe]**.
- An override such as `elems=[['Mg', 0.3]]` means A(Mg) = A⊙(Mg) + [M/H] + 0.3, which is [Mg/M] = 0.3.
- The two differ as soon as Fe gets its own value. That happens when you set it (`elems=[['Fe', 0.1]]`), or when Fe is fitted line by line and `iterate` feeds the measured [Fe/M] back in.
- The results also give `xh` = [X/H] = [X/M] + [M/H], with [M/H] the star's value (from FERRE).

When Fe has its own value:

[X/Fe] = [X/M] − [Fe/M] = [X/H] − [Fe/H]

In code, that is `res.summary['X']['xh'] - res.summary['Fe']['xh']`.

1. Installation

turbofit is plain Python on top of three external pieces:

- **Turbospectrum**: always needed.
- **The NLTE data**: only for NLTE mode.
- **FERRE**: only for stellar-parameter fitting.

1.1 Turbospectrum v20

```
git clone https://github.com/bertrandplez/Turbospectrum_NLTE
cd Turbospectrum_NLTE/exec-gf && make        # gfortran; or exec/ with ifx/ifort
```

Then set two environment variables, for example in your `~/.bashrc`:

```
export TURBOSPEC_DIR=/path/to/Turbospectrum_NLTE
export PATH=$TURBOSPEC_DIR/exec-gf:$PATH      # puts babsma_lu and bsyn_lu on PATH
```

turbofit uses `$TURBOSPEC_DIR/DATA` for Turbospectrum's data files (molecular equilibrium, partition functions, H lines). If `TURBOSPEC_DIR` is not set, it falls back to a `data/` folder or symlink next to the code.

> **IMPORTANT**
> Turbospectrum is driven through small **csh** scripts, so `csh` (or `tcsh`) must be installed. On clusters it almost always is; on a fresh Linux machine use `apt install tcsh` or similar.

Check the installation by synthesising one short spectrum, as in chapter 2, before doing anything large.

1.2 Python packages

numpy, scipy, astropy and matplotlib. Nothing else is required.

```
pip install numpy scipy astropy matplotlib
```

turbofit uses flat imports, so there is nothing to install:

```
import sys
sys.path.insert(0, '/path/to/turbofit')
import synthesis, broaden, linefit
```

1.3 Model atmospheres

used for	grid	how turbofit reads it
CNO-LTE mode, grids, FERRE	APOGEE MARCS grid with [C/M] and [α/M] nodes	`turbogrid.make_atmod_lookup(griddir, model='marcs', flat=True)` returns a function `lookup(teff, logg, mh, am, cm)` giving the Turbospectrum-ready file for a node
NLTE mode	standard MARCS grid (ASCII web format, `*.mod`, all in one directory)	`nlte.NLTEConfig(marcs_dir=..., ...)` indexes it automatically

`make_atmod_lookup` converts each needed model once and caches the converted copy in `griddir/_turbo_cache`. It also uses the `*.filled` stand-ins written by `atmos.fill_holes()` for missing nodes.

1.4 NLTE data (only for NLTE mode)

Step 1: compile the NLTE interpolator that ships with Turbospectrum.

```
cd $TURBOSPEC_DIR/interpolator
gfortran -o interpol_modeles_nlte interpol_modeles_nlte_gfort.f
# (Intel compilers: interpol_modeles_nlte.f)
```

Step 2: download from https://keeper.mpdl.mpg.de/d/6eaecbf95b88448f98a4/, for each element you want:

- the departure-coefficient grid computed on **MARCS** models: `NLTEgrid*MARCS*.bin` and `auxData*MARCS*.dat`;
- the model atom, `atom.*`.

The grids are large, so download only the elements you need.

Step 3: lay the files out as listed in `nlte_filenames.cfg` (a copy ships with turbofit):

```
nlte_data/
  Mg/NLTEgrid4TS_Mg_MARCS_....bin
  Mg/auxData_Mg_MARCS_....dat
  Ca/...
  model_atoms/atom.mg86b
  model_atoms/atom.ca105b
  nlte_filenames.cfg          (optional; turbofit's copy is used otherwise)
```

Step 4: check what turbofit can see.

```
import nlte
cfg = nlte.NLTEConfig(marcs_dir='/data/marcs_standard', nlte_dir='/data/nlte_data')
print(cfg.available_elements())     # e.g. ['Ca', 'Fe', 'Mg']
```

NLTE needs line lists whose lines carry **NLTE level labels**. The Gaia-ESO NLTE atomic list on the same keeper page has them (4200–9200 Å). VALD extractions do not, but TSFitPy's `utilities/` can add the labels. Lines without labels are simply computed in LTE.

1.5 FERRE (only for stellar parameters)

```
git clone https://github.com/callendeprieto/ferre
cd ferre/src && make          # produces ../bin/ferre.x and ../bin/ascii2bin
```

Pass the path to `ferre.x` as `ferre_exe=` (see chapter 7).

1.6 Quick check

```
import os, sys
sys.path.insert(0, '/path/to/turbofit')
import synthesis, turbogrid

lookup = turbogrid.make_atmod_lookup('/data/marcs_apogee', flat=True)
atm = lookup(4500., 2.0, 0.0, 0.0, 0.0)
f, c, w = synthesis.synthesize(4500., 2.0, wrange=[15200, 15210], dw=0.01,
                               atmod=atm, linelists=[...])
print(len(w), (f / c).min())          # ~1001 pixels and some absorption
```

2. Synthesising spectra

2.1 One spectrum: `synthesis.synthesize()`

```
import synthesis, turbogrid
lookup = turbogrid.make_atmod_lookup('/data/marcs_apogee', flat=True)
atm = lookup(4500., 2.0, -0.5, 0.25, -0.25)          # teff, logg, [M/H], [a/M], [C/M]

flux, cont, wave = synthesis.synthesize(
    4500., 2.0, mh=-0.5, am=0.25, cm=-0.25, nm=0.5, vmicro=1.8,
    elems=[['Al', -0.2]],                 # optional [X/M] overrides
    wrange=[15150., 15250.], dw=0.01,
    atmod=atm, linelists=['atomic.list', 'molecular.list', 'Hlinedata'])
norm = flux / cont
```

It returns three arrays on Turbospectrum's wavelength grid:

- `flux`: absolute flux;
- `cont`: continuum;
- `wave`: wavelength in Å.

What each argument controls

argument	meaning
`teff, logg`	used for the geometry choice and the default isotope ratios (the atmosphere itself comes from `atmod`)
`mh, am, cm, nm`	abundances: [M/H] scales every element heavier than He; [α/M] adds to O, Ne, Mg, Si, S, Ar, Ca, Ti; [C/M] and [N/M] add to C and N
`elems`	`[[element, [X/M]], ...]` sets these elements to exactly that [X/M] (overriding `am`/`cm`/`nm`)
`vmicro`	microturbulence in km/s, passed to Turbospectrum as a fixed value (`XIFIX`)
`wrange, dw`	wavelength range and step (Å)
`atmod, atmos_type`	atmosphere file; type auto-detected (`'marcs'`, `'kurucz'`, …). For Turbospectrum-format interpolated models pass any type other than `'marcs'`, e.g. `'interpol'`
`dospherical`	spherical radiative transfer for MARCS models with log g ≤ 3 (default `True`)
`solarisotopes`	`None` (default) picks from the model name: `p…` plane-parallel → solar ratios, `s…` spherical → giant ratios ($^{12}C/^{13}C = 15$); otherwise from log g
`segments`	list of `(wmin, wmax)`: compute only these windows (see below)
`nlte, nlte_elems`	NLTE mode, see chapter 3
`workdir, save`	where Turbospectrum runs (default: a temporary directory, deleted afterwards unless `save=True`)

The abundance array Turbospectrum receives is built by `synthesis.make_abundances(mh, am, cm, nm, elems)`. Call it yourself to see exactly which A(X) are used.

Only some windows: `segments`

When you only need a few windows, synthesising the whole range is wasteful:

```
f, c, w = synthesis.synthesize(..., wrange=[15100, 16900],
                               segments=[(15739.5, 15742.5), (15765.0, 15768.5)])
```

Turbospectrum then computes just those windows, using its `SEGMENTSFILE`. The returned arrays hold the windows one after another.

> **NOTE**
> Within segments, Turbospectrum samples at R = 500 000, its default, and `dw` is ignored there. turbofit does not set the `RESOLUTION` keyword because Turbospectrum v20.1 reads it incorrectly. 500 000 is finer than any instrument, and everything downstream resamples anyway.

2.2 Many abundance patterns for one star: `abund.py`

`abund.synthesize_sets()` uses one atmosphere and returns one spectrum per abundance pattern:

```
import abund
wave, fluxes, conts = abund.synthesize_sets(
    4500., 2.0, mh=-0.5, am=0.25, cm=-0.25, nm=0.5, vmicro=1.8,
    abund_sets=[[],                                    # the base star
                [['Mg', 0.5]],
                [['Mg', 0.5], ['Al', -0.3]]],
    wrange=[15150., 15250.], dw=0.01, atmod=atm, linelists=ll)
# fluxes.shape == (3, npix)
```

Each entry of `abund_sets` is applied on top of `elems`.

`abund.ladder()` is the common special case: one element, stepped around its starting value.

```
xm, wave, fluxes, conts = abund.ladder(4500., 2.0, 'Mg', offsets=[-0.4, -0.2, 0, 0.2, 0.4],
                                       mh=-0.5, am=0.25, wrange=[15150., 15250.], dw=0.01,
                                       atmod=atm, linelists=ll)
# xm = [X/M] values actually used: base [Mg/M] (= am here) + offsets
```

`abund.base_xm(elem, am, cm, nm, elems)` gives the starting [X/M] of an element: `am` for α elements, `cm` for C, `nm` for N, the `elems` value if one is given, and 0 otherwise.

> **WARNING**
>
> **Continuum opacities are recomputed for every set by default (`rerun_babsma=True`).** bsyn takes the electron pressure from babsma's opacity file. Reusing one file across abundance sets therefore freezes the ionisation balance. In a 4500 K giant, ±0.3 dex in Fe gave normalised-flux errors of **0.09**, and Ti and Ni gave 0.001–0.003. Set `rerun_babsma=False` only for trace elements and small steps; it is about 25 % faster.

NLTE works here too. If the varied element is in `nlte_elems`, its departure coefficients are re-interpolated for each abundance step.

2.3 Typical run times

As a rough guide (gfortran build, one core, Turbospectrum v20.1, a 4500 K giant with a VALD atomic list):

- about 2.5 s for a 50 Å range at `dw = 0.01` (babsma + bsyn);
- about 0.8 s for two 1–2 Å segments.

Molecule-rich H-band lists are slower per Å.

3. NLTE

3.1 When to use it

turbofit has two ways of synthesising an element:

	CNO-LTE	NLTE
atmospheres	APOGEE MARCS grid with [C/M] and [α/M] nodes	standard MARCS grid (α tied to [Fe/H], solar C/N)
radiative transfer	LTE	NLTE for the chosen elements, LTE for everything else
between grid points	spectra are interpolated (never atmospheres)	Turbospectrum's interpolator builds the atmosphere *and* the departure coefficients at the exact parameters
best for	Teff, log g, [M/H] (where CNO atmospheres matter); C, N, O from molecules; elements without NLTE data	atomic abundances with strong NLTE effects

The two are kept separate on purpose. The public departure-coefficient grids were computed on standard MARCS models, so applying them to the CNO atmospheres would be inconsistent. The usual chain is therefore:

1. stellar parameters from FERRE on a CNO grid (LTE);
2. abundances in NLTE wherever data exist, and CNO-LTE otherwise.

Every abundance turbofit reports records which mode was used.

NLTE data exist for H, O, Na, Mg, Al, Si, Ca, Ti, Cr, Mn, Fe, Co, Ni, Sr, Y, Ba and Eu. Molecules (CO, CN, OH, CH) are always in LTE in Turbospectrum.

3.2 Setting it up

```
import nlte
cfg = nlte.NLTEConfig(marcs_dir='/data/marcs_standard',     # *.mod, ASCII web format
                      nlte_dir='/data/nlte_data')           # bins, aux files, model_atoms/
cfg.available_elements()                                     # what is actually on disk
```

Optional arguments:

- `config_file`: which `nlte_filenames.cfg` to use;
- `model_atom_dir`: default `nlte_dir/model_atoms`;
- `interpol_dir`: default `$TURBOSPEC_DIR/interpolator`;
- `comp`: MARCS composition class, default `'st'` (standard).

3.3 One NLTE spectrum

```
f, c, w = synthesis.synthesize(4500., 2.0, mh=-0.5, vmicro=1.8,
                               wrange=[5500., 5600.], dw=0.01, linelists=ll,
                               nlte=cfg, nlte_elems=['Mg', 'Ca'])
```

`atmod` is not needed, and is ignored if given. `am` and `cm` still change the *synthesis* abundances, but not the atmosphere, which is standard MARCS.

If a requested element has no data, it is computed in LTE with a warning. If none of the requested elements has data, an error is raised.

3.4 What happens under the hood

For each call, `nlte.prepare()` does the following:

1. **Picks the model family.** For log g < 3 it uses spherical, 1 M☉, ξ = 2 km/s models. For log g ≥ 3 it uses plane-parallel ξ = 1 km/s models, or ξ = 2 if those are missing. This choice only concerns the atmosphere *structure*: the microturbulence used in the synthesis is still your `vmicro`.
2. **Finds the 8 MARCS models bracketing (Teff, log g, [M/H]).** If a corner model is missing, the face of the cube with the most missing corners is moved out by one grid step until all 8 exist, as TSFitPy does.
3. **Runs `interpol_modeles_nlte` once per NLTE element.** It interpolates the atmosphere and that element's departure coefficients with the same weights, at the element's A(X). It writes `*.interpol` (the model) and `*_<El>_coef.dat` (the departures).
4. **Cleans the departure files.** NaNs in the deepest layer become 1 (LTE); other NaNs take the value of the layer below.
5. **Writes the NLTE species file** that bsyn reads (`'NLTE :' '.true.'`, `'NLTEINFOFILE:'`).

The interpolator inputs, species file and NaN handling follow TSFitPy (Gerber et al. 2023; Storm & Bergemann 2023).

3.5 NLTE in grids and line fits

- **Grids:** `turbogrid.write_grid_table(..., nlte=cfg, nlte_elems=[...])` with `atmod_lookup=turbogrid.nlte_atmod_lookup`. See chapter 5.
- **Line by line:** give `LineFitter(nlte=cfg, nlte_elems=[...])`. Those elements are then fitted in NLTE when their data are present, and in CNO-LTE otherwise. See chapter 8.

> **CAUTION**
> The NLTE path was built and checked piece by piece (bracketing, NaN clean-up, bsyn reading an NLTE species file), but it has **not yet been run end to end with real departure grids**. Before relying on it, compare one NLTE spectrum with TSFitPy for the same star and line.

4. Broadening, LSFs and resampling

Everything between a raw Turbospectrum spectrum and something comparable with data goes through `broaden.py`. The same code is used by the grids, the RV code and the line fits, so they always agree.

4.1 The one call you need: `broaden.convolve()`

```
import broaden
wave_out, flux_out = broaden.convolve(wave, flux,
        R=22500.,                 # instrument (see 4.2)
        rv=-45.3,                 # km/s, applied to the model
        vmac=4.0, vsini=0.0,      # stellar broadening (km/s)
        dlogw_out=6e-6)           # output grid (see 4.3)
```

`flux` can be one spectrum `(npix,)` or many `(nspec, npix)`. The same matrix is then applied to every row, so convolving a whole grid needs only one matrix build.

The steps, in order:

1. **Stellar broadening** (`broaden.stellar`):
   - Gaussian macroturbulence with profile $\exp(-(v/vmac)^2)$, so $\sigma = vmac/\sqrt{2}$;
   - rotation with Gray's kernel (limb darkening `eps = 0.6`).
2. **Instrumental LSF**, the Doppler shift and resampling, all in **one sparse matrix** (`broaden.lsf_matrix`). The LSF is evaluated at each *output* pixel's own wavelength, which is what makes a varying R or a per-pixel LSF exact.

4.2 Describing your instrument

your instrument	arguments
constant resolving power	`R=22500.` (Gaussian, FWHM = λ/R)
resolving power varying with wavelength	`R=(wave_R, R_values)` (interpolated) or `R=lambda lam: ...`
your own kernel shape	`lsf=lambda dlam, lam: weights, lsf_halfwidth=2.0` (Å)
a per-pixel LSF array (APOGEE, NIRSpec, ...)	`lsf=array` of shape `(n_output_pixels, n_k)` and `lsf_offsets` of length `n_k`, in *output pixels*
detector pixel not negligible vs LSF	add `pixel_integrate=True` (averages the kernel over each output pixel)

Kernels are normalised automatically.

> **TIP**
> Sample LSF arrays finely, ≤ 0.2 pixel (e.g. `lsf_offsets=np.arange(-7, 7.01, 0.1)`), because the kernel is linearly interpolated between offsets. At 1-pixel sampling, a σ ≈ 1-pixel LSF is widened by about 8 %. A fitted vmac absorbs this, and abundances are unaffected because equivalent widths are conserved, but vmac is then biased low.

4.3 Choosing the output grid

Give exactly one of:

- `wave_out=array`: e.g. the observed pixels. This is what the fitting code uses.
- `dw_out=0.1`: a linear grid in Å.
- `dlogw_out=6e-6`: $\log_{10}$-uniform, like APOGEE/ASPCAP pixels.

With none of them, the output stays on the input grid (shifted if `rv` is set). A useful rule of thumb is at least 2–3 output pixels per resolution element: $\lambda/(R\cdot\Delta\lambda) \geq 2$.

4.4 Doppler shifts

`broaden.doppler(wave, rv)` uses the relativistic formula $\lambda_obs = \lambda\cdot\sqrt{((1+\beta)/(1-\beta))}$. With `rv` given, the model is shifted to the observed frame *before* the LSF is applied, so the LSF stays defined in the detector frame.

4.5 How accurate is it?

The following were checked numerically:

- **Line width:** the measured σ of a convolved line is 0.29027 Å against 0.29028 Å expected.
- **Equivalent width** is conserved to a few parts in 10^9.
- **R(λ):** a constant R(λ) matches a scalar R exactly.
- **LSF arrays:** a finely sampled Gaussian LSF array matches R to 8×10^{-5} in flux.
- **RV shift:** a 50 km/s shift is recovered to 0.004 km/s.

5. Building grids for FERRE

`turbogrid.py` builds Synple-style grids with Turbospectrum. The whole workflow is described step by step in `turbogrid_user_guide.pdf`:

1. grid table;
2. Slurm or GNU parallel jobs;
3. collect;
4. fill holes;
5. convolve;
6. write the FERRE grid.

This chapter covers what is new in turbofit and gives the short version.

5.1 The short version

```
import numpy as np, turbogrid as tg

lookup = tg.make_atmod_lookup('/data/marcs_apogee', model='marcs', flat=True)
params = tg.make_grid_params(teff_grid=(15, 3000., 250.), logg_grid=(12, -0.5, 0.5),
                             mh_grid=(14, -2.5, 0.25), cm=(11, -1.5, 0.25),
                             am=(7, -1.0, 0.25), nm=(6, -0.5, 0.5),
                             vmicro='holtzman', atmod_lookup=lookup)

tg.write_grid_table(params, outdir='gridm31', wrange=(15050., 17050.), dw=0.01,
                    linelists=ll, atmos_type='marcs',
                    outputs_dir='/scratch/me/m31grid', shard_size=1000,
                    broaden_kw=dict(R=22500., dlogw_out=6e-6))      # new, see 5.2
tg.write_slurm_array('gridm31', jobname='m31grid', time='24:00:00', mem='8G',
                     partition='nodes', array_step=1011, max_concurrent=30,
                     max_array_size=999)
#   sbatch gridm31/submit_array.slurm      ... then:
tg.collect_grid('gridm31', gridfile='/scratch/me/m31grid/grid.npz')
#   python fill_grid_holes.py /scratch/me/m31grid/grid.npz --labels teff logg mh cm am nm ...
tg.write_ferre_grid('/scratch/me/m31grid/grid_filled.npz', 'gridm31/grid.ferre',
                    labels=('teff', 'logg', 'mh', 'cm', 'am', 'nm'),
                    resolution=22500, vacuum=1)
```

5.2 Broadening on the fly with any LSF: `broaden_kw`

`write_grid_table(..., broaden_kw=dict(...))` passes the dictionary straight to `broaden.convolve()` right after each spectrum is synthesised. Only the broadened spectrum is stored. Anything from chapter 4 works:

```
broaden_kw = dict(R=22500., dlogw_out=6e-6)                          # constant R, log-λ pixels
broaden_kw = dict(R=(wave_R, R_of_wave), dw_out=0.1)                 # R varying with λ
broaden_kw = dict(lsf=lsf_array, lsf_offsets=np.arange(-7, 7.01, 0.1),
                  wave_out=instrument_wave)                           # per-pixel LSF
```

The older `R_out`, `dw_out` and `wout` options still work. `broaden_kw` takes priority if both are given.

> **TIP**
> Make `wrange` a few resolution elements wider than the range you want in the output, so convolution edge effects fall outside it.

5.3 NLTE grids

```
import nlte
cfg = nlte.NLTEConfig('/data/marcs_standard', '/data/nlte_data')
params = tg.make_grid_params(teff_grid=..., logg_grid=..., mh_grid=..., am=..., nm=...,
                             vmicro='holtzman', atmod_lookup=tg.nlte_atmod_lookup)
tg.write_grid_table(params, outdir='grid_nlte', ..., nlte=cfg, nlte_elems=['Mg', 'Ca', 'Fe'])
```

`nlte_atmod_lookup` tells turbogrid that each atmosphere will be interpolated from the standard MARCS grid when the point runs. `am` and `cm` then only change the synthesis abundances.

5.4 FERRE grids on log-λ pixels

`write_ferre_grid()` now detects the wavelength grid:

- linear grids get `LOGW = 0` and `WAVE = λ₀ Δλ`;
- log₁₀-uniform grids (e.g. from `dlogw_out`) get `LOGW = 1` and `WAVE = log₁₀λ₀ Δlog₁₀λ`.

5.5 Axis order (verified)

In the flux block, **the last label in `labels` varies fastest**. This was checked with FERRE itself: pure interpolation at every node of a test grid returned that node's spectrum to within 4×10^{-4}. Use the same `labels` order in `fill_grid_holes.py` and `write_ferre_grid()`.

> **NOTE**
> Set `vacuum=1` if your line lists are in vacuum wavelengths (APOGEE lists are). The flag should describe your line lists, not Turbospectrum's default.

6. Radial velocities

`rv.py` measures the velocity of a normalised spectrum against a synthetic template. The template should be at the instrument's resolution, for example a grid spectrum or `broaden.convolve()` output.

6.1 Cross-correlation, then χ^2

```
import rv
v0, e0, (vgrid, ccf) = rv.ccf_rv(wave, flux, twave, tflux, err=err,
                                 vmin=-600., vmax=600.)
v, e, (vv, chi2) = rv.chi2_rv(wave, flux, err, twave, tflux, v0)
```

`ccf_rv` works as follows:

- Both spectra are put on a common log-λ grid (step 1/3 of an observed pixel), the weighted, normalised cross-correlation is computed over `vmin..vmax`, and a parabola refines the peak.
- The uncertainty follows Zucker (2003), with N set to the number of *observed* pixels used.
- A `mask` (True = use the pixel) excludes tellurics or bad regions.
- A peak at the edge of the search range gives `rv_err = nan`. Widen the range if that happens.

`chi2_rv` scans around the CCF velocity, fits a parabola to $\chi^2(v)$, and returns the velocity where χ^2 is lowest. Its error is where χ^2 rises by 1, scaled by the reduced χ^2 when that is above 1. It is the recommended final step.

6.2 How well calibrated are the errors?

On 30 simulated spectra (R = 22 500, S/N = 100, random velocities within ±300 km/s), the pulls (v − v_true)/σ had:

- a mean of 0.02 and a standard deviation of 0.88 for `ccf_rv`;
- a mean of 0.02 and a standard deviation of 0.76 for `chi2_rv`.

Both error estimates are therefore realistic to slightly conservative.

6.3 Per-line RV offsets

The line-by-line fitter also fits a small velocity offset per line, within ±`drv_max` km/s (default 5). This absorbs wavelength-calibration and line-list errors, and the offsets are reported as `drv` in the results table.

7. Stellar parameters with FERRE

`ferre.py` is a thin driver for FERRE (Allende Prieto et al. 2006) on grids written by `turbogrid.write_ferre_grid()`. It:

1. resamples spectra onto the grid's pixels in the rest frame;
2. writes the FERRE inputs;
3. runs FERRE;
4. reads the results back.

7.1 Fitting

```
import ferre
out = ferre.fit([wave1, wave2], [flux1, flux2], [err1, err2], rv=[-121.4, 33.0],
                grid='gridm31/grid.ferre', ids=['star1', 'star2'],
                workdir='ferre_run', ferre_exe='/path/to/ferre.x', nthreads=8)
r = out['star1']
r['params']      # {'TEFF': 4521., 'LOGG': 1.93, 'METALS': -0.41, 'C': ..., 'ALPHA': ..., 'N':
...}
r['errors']      # same keys, 1-sigma
r['chi2']        # reduced chi^2
r['cov']         # covariance matrix (grid-label order)
r['model']       # best-fitting model on the grid pixels (r['wave'])
r['edge']        # labels within STEPS/8 of a grid edge
r['failed']      # True if FERRE flagged the fit as failed
```

Spectra must be continuum-normalised in the same way as the grid (the grid is normalised by default). Pixels outside your data, or marked bad, get an error of 10^{10}, so FERRE ignores them.

7.2 Controlling FERRE

Default namelist:

keyword	value	meaning
`NDIM`, `NOV`, `INDV`	all grid dimensions	fit every parameter
`INTER`	`min(3, min(N_P) − 1)`	cubic Bézier where the grid allows
`ALGOR`	3	Nelder–Mead
`ERRBAR`, `COVPRINT`	1, 1	errors and covariance
`INIT`, `INDINI`, `NRUNS`	1, 1…, 1	start from the `.ipf` values (default: grid centre)
`F_FORMAT`, `F_ACCESS`	0, 0	ASCII grid loaded into memory

Anything else goes through `extra=`, which overrides or adds keywords:

```
ferre.fit(..., extra={'CONT': 1, 'NCONT': 0,       # FERRE's own continuum renormalisation
                      'INDINI': '2 2 2 2 2 1', 'NRUNS': 32})
```

Starting values can be given per star as `init=[{'TEFF': 4500., 'LOGG': 2.0}, ...]`, using grid-label names. They are nudged 0.001 inside the grid edges, because FERRE rejects starting points that sit exactly on an edge.

7.3 From FERRE to line-by-line fitting

```
star, star_err = ferre.to_star(r, rv=v)
# star     = {'teff':..., 'logg':..., 'mh':..., 'am':..., 'cm':..., 'nm':..., 'vmicro':...,
'rv':...}
# star_err = {'teff':..., 'logg':..., 'mh':..., ...}
```

If the grid has no vmicro axis, `vmicro` comes from the Holtzman et al. (2015) relation, $2.478 - 0.325 \log g$, unless you pass `vmicro=`. `star` and `star_err` go straight into `LineFitter.fit()`.

7.4 The lower-level pieces

- `read_header(grid)` parses the `&SYNTH` header.
- `grid_wave(hdr)` gives the pixel wavelengths.
- `to_grid_pixels(wave, flux, err, rv, hdr)` resamples one spectrum.
- `write_inputs(workdir, ids, flux, err, hdr, init, nthreads, extra=...)` writes the input files.
- `read_output(workdir, ids, hdr)` reads the results.

7.5 Three FERRE behaviours to know about

- FERRE ends with `STOP 1`, a **non-zero exit status, even after a normal run**. turbofit judges success by the output file instead of the exit code.
- With `COVPRINT=1`, the current FERRE writes the **full n×n covariance**, not the upper triangle described in the manual. turbofit reads either.
- The output parameter file holds the id, n parameters, n errors, then three more columns (fraction, S/N, $\log_{10}$ reduced χ^2), then the covariance.

8. Line-by-line abundances

`linefit.py` measures abundances window by window for a star whose stellar parameters are already known, usually from FERRE. For each window it fits:

- **[X/M]**: the abundance;
- **ΔRV**: a small velocity offset;
- **vmac**: macroturbulent broadening;
- **a local continuum**: a polynomial per window.

Each abundance comes with a statistical error and a systematic error propagated from the stellar-parameter uncertainties.

8.1 Setting up a `LineFitter`

```
import turbogrid, linefit, nlte

lf = linefit.LineFitter(
    linelists=['atomic.list', 'molecular.list', 'Hlinedata'],
    inst=dict(R=22500.),                                  # any instrument from chapter 4
    # CNO-LTE: the atmosphere nodes and how to find a node's file
    nodes=dict(teff=(15, 3000., 250.), logg=(12, -0.5, 0.5), mh=(14, -2.5, 0.25),
               am=(7, -1.0, 0.25), cm=(11, -1.5, 0.25)),
    atmod_lookup=turbogrid.make_atmod_lookup('/data/marcs_apogee', flat=True),
    # NLTE for these elements wherever data exist (optional)
    nlte=nlte.NLTEConfig('/data/marcs_standard', '/data/nlte_data'),
    nlte_elems=['Mg', 'Al', 'Ca'],
    n_jobs=16)                                            # parallel Turbospectrum runs
```

Node axes can be `(n, start, step)` triplets, as in turbogrid, or explicit arrays.

option	default	meaning
`offsets`	−0.6 … +0.6 in 0.2 dex steps	abundance ladder around the starting [X/M]; extended automatically if a fit reaches its end (`max_extend=2` times)
`drv_max`	5 km/s	bound on the per-line velocity offset
`vmac`, `vsini`	3.0, 0.0 km/s	starting / fixed stellar broadening (see 8.5)
`cont_deg`	1	continuum polynomial per window (0 = scale only, `None` = none)
`dw`	0.02 Å	synthesis step for the full-range runs in `automask`
`workdir`	temporary	where the Turbospectrum runs happen

`lf.mode('Mg')` tells you which mode an element will use.

8.2 The two modes

CNO-LTE is used for every element not handled in NLTE.

- turbofit finds the cell of the atmosphere grid around the star: $2^5 = 32$ nodes in (Teff, log g, [M/H], [α/M], [C/M]).
- It synthesises each element's abundance ladder at every node, then interpolates the *spectra* multilinearly to the star's parameters.
- Atmospheres are never interpolated.
- Every node is synthesised with the **star's** abundances, so only the atmospheric structure is interpolated, not the line strengths of other species.
- A star exactly on a node still gets a full cell. The neighbour node carries zero weight but provides the derivatives used for the systematics.
- A missing node atmosphere raises an error that names the missing corners. Fill the grid with `atmos.fill_holes()` first.

NLTE is used for elements listed in `nlte_elems` that have data.

- Turbospectrum's interpolator gives the atmosphere and the departure coefficients at the exact parameters (chapter 3).
- There is one synthesis per abundance step and no node interpolation.

In both modes only the line windows are synthesised (Turbospectrum segments). Every fit iteration afterwards is pure interpolation plus broadening, with no Turbospectrum calls.

8.3 Line masks

Your own

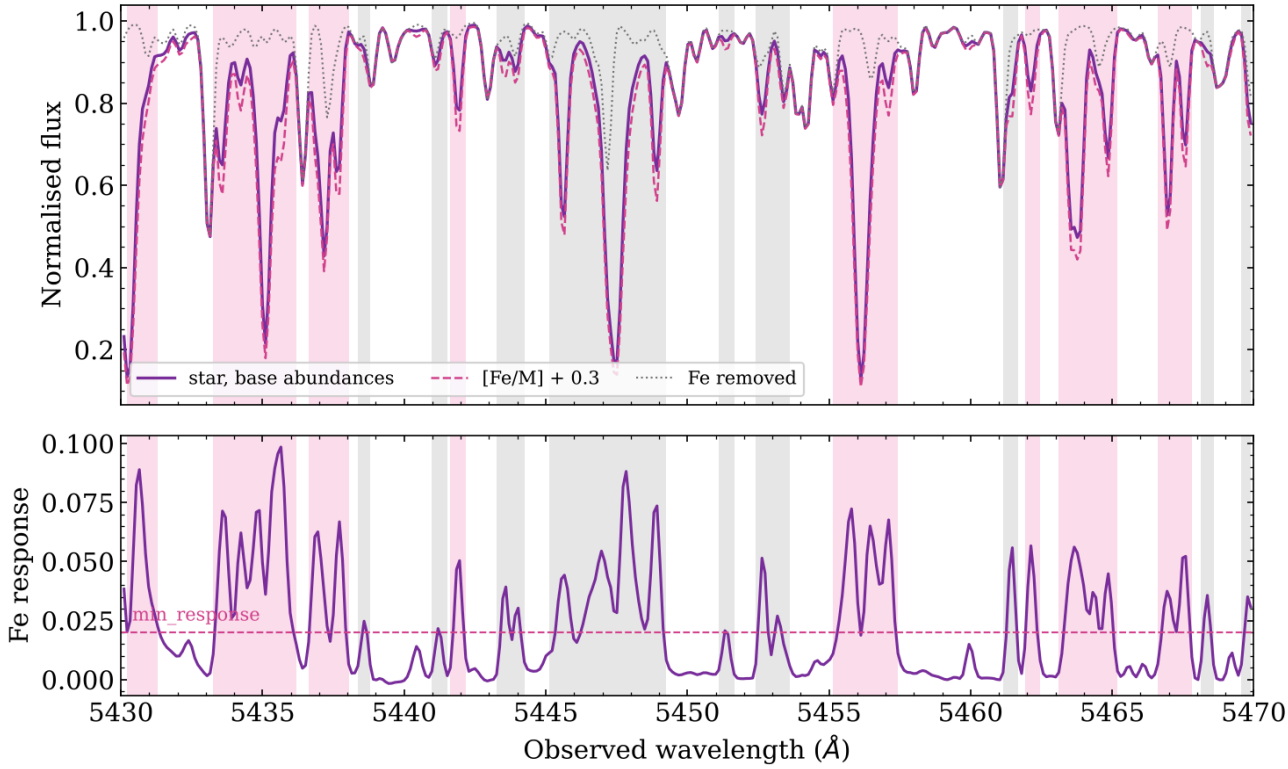
```
mask = linefit.read_mask('mg_lines.txt')                    # turbofit format
mask = linefit.read_mask('fe-lmask.txt', element='Fe')     # TSFitPy format (; comments)
```

turbofit format, one window per line, **rest-frame** wavelengths in the same convention as your line lists:

```
# element  center  wmin  wmax  response  blend
Mg   15740.716  15739.900  15741.500   0.0810  0.050
C    15578.400  15577.950  15578.900   0.0420  0.180
```

Automatic: built for each star and instrument

```
mask = lf.automask(['C', 'N', 'O', 'Mg', 'Al', 'Fe'], star, wave,
                   dx=0.3, min_response=0.02, max_blend=0.3, cut=0.25,
                   exclude=[(15890., 15960.)])     # observed-frame ranges to avoid
linefit.write_mask(mask, 'star1_mask.txt')          # inspect / edit / reuse
```



How `automask` chooses windows (Fe, 4500 K giant, R = 20 000).

- **Top:** the star (purple), with [Fe/M] + 0.3 (pink dashed) and with Fe removed (grey dotted).
- **Bottom:** the Fe response.
- **Shading:** pink windows are kept; grey windows respond to Fe but are dominated by other species.

What it does:

1. Synthesises the star over the observed range for three cases: the base abundances, [X/M] + `dx` for each element, and each element removed. It uses the nearest atmosphere node, or NLTE if no nodes are configured.
2. Broadens everything with *your* instrument and resamples onto *your* pixels at the star's RV.
3. Computes the response, r = F_base − F_+dx, and finds peaks above `min_response` at least one FWHM apart.
4. Grows each window until r falls below `cut` × the peak.
5. Measures the **blend fraction**: the response-weighted share of the line depth that comes from *other* species. This uses the spectrum with X removed. Windows with blend > `max_blend` are dropped.
6. Merges overlapping windows and records `response` and `blend` for each.

This works the same way for molecular features (C from CO/CN/CH, N from CN, O from OH) as for atomic lines.

> **TIP**
>
> `min_response` is a flux change for +`dx` dex. A sensible value is a few times your noise per pixel, divided by √(pixels per line). Low-S/N spectra want a higher threshold. Lines that end up too weak are flagged `weak` anyway (8.8).

8.4 Fitting

```
star = dict(teff=4520., logg=1.9, mh=-0.42, am=0.18, cm=-0.20, nm=0.30,
            vmicro=1.6, rv=-121.4)
star_err = dict(teff=50., logg=0.10, mh=0.05, am=0.03, cm=0.05)

res = lf.fit(wave, flux, err, star, mask=mask, star_err=star_err,
             vmac='global', iterate=1, profile=True, mc=0)
```

If `mask` is omitted, `elements=[...]` must be given and `automask` is run once.

The model for each window is built in this order:

1. The interpolated ladder is turned into a continuous function of [X/M] with a cubic spline.
2. Stellar broadening is applied.
3. The LSF, the Doppler shift (star RV + ΔRV) and resampling onto the observed pixels are applied together.
4. The model is multiplied by a continuum polynomial, which is solved linearly at every step.

The fit uses `scipy.optimize.least_squares`, with [X/M] bounded by the ladder. When a fit reaches the ladder's end, the ladder is extended and the window refitted.

Measurement errors: see 8.6 below.

Blends between fitted elements: `iterate`

Pass 1 fits every element with the starting abundances. Each extra pass (`iterate=1` gives two passes in total) feeds the measured means back into the synthesis as `star['elems']`, so a Ti line blending an Fe window is modelled at the measured Ti abundance. Each pass re-synthesises the ladders.

Known abundances of other species can also be given directly:

```
star['elems'] = [['Ce', 0.2], ['Nd', 0.3]]
```

8.5 Macroturbulence: `vmac=`

value	behaviour
`'global'` (default)	a first pass fits vmac per line; it is then fixed to the median of the good lines, weighted by `snr_resp`
`'line'`	free for every line (beware: degenerate with abundance for weak lines in narrow windows)
a number	fixed

The value used is stored in `res.vmac`.

8.6 Measurement errors on the abundances

There are three per-line estimates of the measurement (noise) error on [X/M], and a combined error for each element.

column	how it is computed	cost
`err_stat`	curvature of χ^2 at the best fit (from the Jacobian), scaled by $\sqrt{\chi^2}$_red when χ^2_red > 1	free
`err_lo`, `err_hi`	**χ^2 profile**: [X/M] is stepped across the best fit, with the RV offset (and vmac, if free) re-optimised at each step. The errors are where χ^2 rises by 1 (in units of χ^2_red when that exceeds 1). They can be asymmetric, which matters for weak lines and lines on the saturated part of the curve of growth. `inf` means χ^2 never rose by 1 within the abundance ladder, so the line does not constrain the abundance on that side.	~17 small refits per line; on by default (`profile=True`)
`err_mc`	**Monte Carlo**: the line is refitted `mc` times with Gaussian noise drawn from your `err` added to the data. This is the standard deviation of those fits.	`mc` refits per line; off by default (`mc=0`; `mc=100` is plenty)

None of these need extra Turbospectrum runs; they only refit the interpolated model. They are computed on the final pass only. Pass `seed=` for reproducible Monte Carlo errors.

These errors describe the noise in your spectrum, so they are only as good as your `err` array. If the per-pixel errors are underestimated, χ^2_red > 1 and `err_stat`/`err_lo`/`err_hi` are inflated accordingly. `err_mc` is not, so compare it with them as a check.

For each element (`res.summary`):

key	meaning
`err_mean`	error of the weighted mean from the per-line `err_stat`
`std`, `err_scatter`	line-to-line standard deviation, and `std/√n`
`err_meas`	measurement error of the mean: the larger of `err_mean` and `err_scatter`
`err_sys`	stellar-parameter systematic (8.7), correlated between lines
`err_total`	√(`err_meas`² + `err_sys`²)

Using the larger of the propagated error and the observed scatter means the quoted error of the mean also covers line-to-line problems the noise model does not know about (log gf errors, unmodelled blends, continuum placement).

Calibration test. One simulated spectrum (4500 K giant, R = 20 000, S/N = 100, [Fe/M] = +0.15) was refitted with 12 independent noise realisations. For six Fe windows, the spread of the fitted [Fe/M] values compares with the error estimates as follows:

window (Å)	true scatter	`err_stat`	`err_lo`	`err_hi`	`err_mc` (40 draws)
5405.48	0.016	0.015	0.015	0.015	0.013
5414.03	0.096	0.155	0.156	0.155	0.135
5417.06	0.131	0.141	0.135	0.144	0.147
5423.78	0.043	0.044	0.045	0.044	0.043
5425.30	0.204	0.150	0.147	0.152	0.155
5435.07	0.019	0.020	0.020	0.020	0.019

With 12 realisations the "true scatter" is itself uncertain by about 20 %. The strong and moderate lines agree to within that. The two weakest lines (5414.03 and 5425.30, flagged `weak` at this S/N) differ by up to ~40 % in either direction, because their errors are large and the χ^2 surface is far from parabolic.

8.7 Systematic errors from the stellar parameters

In **CNO-LTE** mode these cost no extra synthesis. The node spectra are already computed, so for each parameter p in `star_err`:

1. the node weights are recomputed at p ± σ_p, staying within the cell;
2. the abundance is refitted with everything else held fixed;
3. the derivative d[X/H]/dp is stored.

The systematic error on [X/M] combines the terms in quadrature, using d[X/M]/d[M/H] = d[X/H]/d[M/H] − 1. Element means treat these errors as **correlated** between lines: the mean derivative is propagated, rather than averaging per-line errors.

These derivatives cover the effect of the *atmospheric structure*. Blending species that scale with [M/H] are held at the star's values.

Optional extras, which need extra syntheses:

- `sys_vmicro=True`: adds vmicro, from ladders at vmicro ± σ for the dominant node only (two extra ladders per element).
- `sys_nlte=True`: in NLTE mode, ladders at Teff, log g and [M/H] ± σ (and vmicro). That is six to eight extra ladders per element.

8.8 Results

```
res.summary['Fe']
# {'n': 7, 'xm': 0.129, 'xh': -0.291, 'err_mean': 0.007, 'std': 0.043,
#  'err_scatter': 0.016, 'err_meas': 0.016, 'err_sys': 0.112, 'err_total': 0.113,
#  'mode': 'CNO-LTE'}
t = res.table()          # astropy Table, one row per window
res.write('star1_lines.csv')
```

column	meaning
`element, center, wmin, wmax`	the window (rest frame)
`mode`	`CNO-LTE` or `NLTE`
`xm`, `xh`	[X/M] and [X/H] (= [X/M] + the star's [M/H]); see below for [X/Fe]
`err_stat`, `err_lo`, `err_hi`, `err_mc`	measurement errors on [X/M] (8.6)
`err_sys`	parameter-propagated systematic error on [X/M] (8.7)
`dxh_d<p>`	d[X/H]/dp for each parameter in `star_err`
`drv`, `vmac`	per-line velocity offset and the vmac used
`chi2r`, `npix`	reduced χ^2 and number of pixels
`snr_resp`	how many σ a 0.1 dex change makes in this window
`response`, `blend`	from the mask
`flags`	`edge` (at the ladder end after extension), `weak` (`snr_resp` < 3), `poorfit` (χ^2_red > 5), `nodata` (< `min_pix` usable pixels)
`ok`	not `edge` and not `weak`; only `ok` lines enter `summary`

Summary means are weighted by 1/err_stat², with a floor of 0.01 dex.

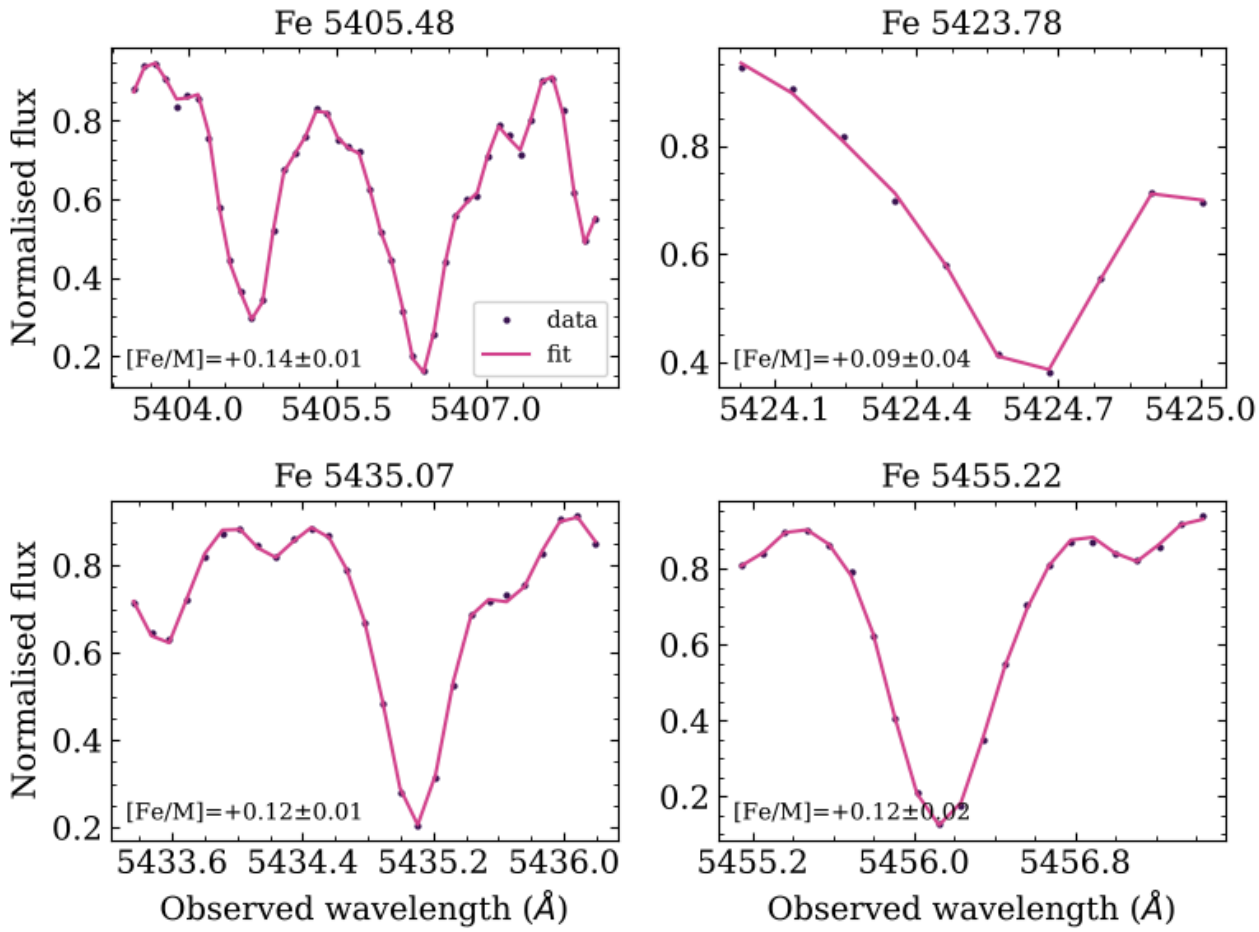
[X/M] versus [X/Fe]. `xm` is relative to the star's overall [M/H]. That equals [X/Fe] only while Fe sits at [Fe/M] = 0. If Fe is among the fitted elements, quote:

```
x_fe = res.summary['Mg']['xh'] - res.summary['Fe']['xh']      # [Mg/Fe]
```

For its error, combine the two statistical errors in quadrature (`err_mean`, or `std`/√n). The [M/H] part of the systematic error largely cancels in the difference, because both elements respond to [M/H] in the same way. The derivatives in `dxh_dmh` show by how much.

8.9 Plots

```
linefit.plot_lines(res, 'star1_lines.pdf', ncols=4, element='Fe',
                   colors=('#3A1450', '#D6458F'))      # data, model; default black/red
```



Fits to four automatically selected Fe windows (simulated R = 20 000, S/N = 150 spectrum with [Fe/M] = +0.15).

8.10 How well does it work?

Tests on simulated spectra (4500 K giant, R = 20 000, VALD atomic lines, 5400–5500 Å):

- **Noise-free, same atmosphere:** [Fe/M] = +0.15 was recovered to **±0.003 dex** line by line.
- **S/N 150, blends from Ti, Ni and Ca at non-solar abundances, vmac fitted:**
  - [Fe/M] = 0.136 ± 0.014 (truth 0.15);
  - [Ti/M] = −0.213 ± 0.021 (truth −0.20);
  - global vmac = 3.7 km/s (truth 4.0).
- **The same fit with a per-pixel LSF array instead of R:** [Fe/M] = 0.139 and [Ti/M] = −0.191.

9. The full pipeline

`pipeline.analyse()` runs the whole chain for one star:

1. **RV template:** FERRE interpolates a spectrum from your grid, at the grid centre or at `template_params`.
2. **RV:** cross-correlation, then χ^2 refinement.
3. **FERRE:** fit on the CNO grid, giving Teff, log g, [M/H], [α/M], [C/M] and [N/M] with errors.
4. **Masks:** `automask` for your elements (or your own `mask=`).
5. **Line by line:** `LineFitter.fit()`, using FERRE's errors as `star_err` for the systematics.

```
import numpy as np, turbogrid, linefit, pipeline

wave, flux, err = np.loadtxt('star1_normalised.txt', unpack=True)
lf = linefit.LineFitter(linelists=ll, inst=dict(R=22500.),
                        nodes=dict(teff=(15, 3000., 250.), logg=(12, -0.5, 0.5),
                                   mh=(14, -2.5, 0.25), am=(7, -1.0, 0.25),
                                   cm=(11, -1.5, 0.25)),
                        atmod_lookup=turbogrid.make_atmod_lookup('/data/marcs_apogee',
flat=True),
                        n_jobs=16)

out = pipeline.analyse(wave, flux, err, grid='gridm31/grid.ferre', lf=lf,
                       elements=['C', 'N', 'O', 'Mg', 'Al', 'Fe'],
                       ferre_exe='/path/to/ferre.x', workdir='star1', nthreads=8,
                       vmac='global', iterate=1)          # extra keywords go to lf.fit

out['rv'], out['rv_err']
out['star'], out['star_err']        # from FERRE
out['ferre']                        # the raw FERRE result (chi2, cov, model, edge flags)
out['mask']                         # the windows used
out['lines'].summary                # abundances per element
```

Everything is written under `workdir`:

- `template/`: the FERRE run that made the RV template;
- `ferre/`: the FERRE fit itself, including the log.

Pass `vmicro=` to override the vmicro taken from the grid or the Holtzman relation, and `rv_range=` to change the velocity search window.

> **NOTE**
> The grid must contain TEFF and LOGG axes for the line-by-line step, because the star dictionary needs them. Any other grid labels (METALS, ALPHA, C, N, VMICRO/LOG10VDOP) are mapped automatically by `ferre.to_star()`.

10. Troubleshooting and known quirks

10.1 Errors you may see

symptom	likely cause	what to do
`Turbospectrum failed` with `babsma_lu: Command not found`	Turbospectrum executables not on `PATH`	`export PATH=$TURBOSPEC_DIR/exec-gf:$PATH`
`/bin/csh: not found` or the scripts do nothing	no csh	install `tcsh`
`FileNotFoundError` for a line list or atmosphere	wrong path	check `linelists`, `atmod`, `griddir`, and `flat=` in `make_atmod_lookup`
bsyn stops with `NDP too small` or `too many wavelengths`	model has more layers than compiled for, or the range/step is too large	split the range, or raise `NDP`/`lpoint` in `source/spectrum.inc` and recompile
`no atmosphere for corner(s) ...` in `linefit`	a node of the cell around the star is missing	fill holes with `atmos.fill_holes()`, or check the star is inside the nodes
`... outside the atmosphere nodes ...`	the star lies beyond the node range	extend the nodes or drop the star
`none of the requested nlte_elems have NLTE data available`	bin/aux/model-atom files not found	`cfg.available_elements()`; check the layout against `nlte_filenames.cfg`
`interpol_modeles_nlte not found`	the NLTE interpolator is not compiled	see 1.4
`NLTE interpolation failed for X`	MARCS models missing around the star, or the abundance is outside the departure grid	check the MARCS directory; departure grids span about −2…+1 dex around solar-scaled
`FERRE failed, see .../ferre.log`	FERRE never wrote its output file	read the log: grid path, `INTER` larger than the axis sizes allow, or start values outside the grid
`LSF narrower than the model sampling`	model step too coarse for the LSF	synthesise on a finer `dw`
`rv_err = nan`	the CCF peak is at the edge of `vmin..vmax`, or the CCF is too flat	widen the range, or check normalisation and template

10.2 Lines flagged in `linefit`

- **`weak`**: a 0.1 dex change moves the window by less than 3σ. The line is uninformative at this S/N. Raise `min_response` in `automask` or accept fewer lines.
- **`edge`**: the fit sits at the end of the abundance ladder even after extension. Usually the window is badly blended, has the wrong continuum, or contains a defect. Look at the

plot.
- `poorfit`: χ^2_red > 5. Typical causes are an unmodelled blend, a bad pixel, or a line-list problem. Check `drv` too: large offsets point to misidentified or blended lines.

10.3 Known quirks (handled for you)

- **Turbospectrum v20.1's `RESOLUTION` keyword** is read with an `(a)` format and gives a garbage value. turbofit never writes it, so segments use Turbospectrum's default R = 500 000.
- **Reusing babsma opacities** across abundance changes freezes the electron pressure (errors up to 0.09 in flux for Fe). babsma is rerun by default (2.2).
- **FERRE ends with `STOP 1`**, a non-zero exit status, even after a successful run. Success is judged by the output file.
- **FERRE's `COVPRINT`** writes the full covariance matrix, not the upper triangle described in the manual.
- **FERRE rejects starting values exactly on a grid edge.** They are nudged 0.001 inside.
- **Departure files sometimes contain NaN.** These are replaced as described in 3.4.

10.4 Getting help

When reporting a problem, include:

- the call;
- the full traceback;
- for Turbospectrum failures, the run directory (`save=True, workdir=...` in `synthesize()` keeps it, including the `*_babsma.log` and `*_bsyn.log` files).

11. API reference

Arguments are listed with their defaults. Full descriptions are in each function's docstring.

synthesis

`synthesize(teff, logg, mh=0.0, am=0.0, cm=0.0, nm=0.0, vmicro=2.0, elems=None, wrange=[15000, 17000], dw=0.1, atmod=None, atmos_type=None, dospherical=True, linelists=None, solarisotopes=None, workdir=None, save=False, verbose=False, nlte=None, nlte_elems=None, segments=None)` Returns `(flux, cont, wave)`.

`make_abundances(mh=0.0, am=0.0, cm=0.0, nm=0.0, elems=None)` Returns the A(X) array for Z = 1…99 (index Z−1).

`do_turbospec(root, atmod, linelists, mh, am, abundances, wrange, dw, save=False, solarisotopes=False, babsma=None, atmos_type='marcs', spherical=True, vmicro=2.0, tfactor=1.0, verbose=False, nlte_info=None, segments=None)` The low-level babsma + bsyn run in the current directory.

abund

`synthesize_sets(teff, logg, mh=0.0, am=0.0, cm=0.0, nm=0.0, vmicro=2.0, abund_sets=([],), elems=None, wrange=(15000, 17000), dw=0.1, atmod=None, atmos_type=None, dospherical=True, linelists=None, solarisotopes=None, segments=None, nlte=None, nlte_elems=None, rerun_babsma=True, workdir=None, save=False)` Returns `(wave, flux[nset, npix], cont[nset, npix])`.

`ladder(teff, logg, elem, offsets=(-0.6, -0.4, -0.2, 0.0, 0.2, 0.4, 0.6), mh=0.0, am=0.0, cm=0.0, nm=0.0, elems=None, **kw)` Returns `(xm, wave, flux, cont)`.

`base_xm(elem, am=0.0, cm=0.0, nm=0.0, elems=None)` The starting [X/M] of an element.

nlte

`NLTEConfig(marcs_dir, nlte_dir, config_file=None, model_atom_dir=None, interpol_dir=None, comp='st')` Methods: `.available_elements()`, `.grid`, `.aux_length(el)`.

`bracket_models(cfg, teff, logg, mh)` Returns `(list of 8 model paths, family)`.

`prepare(cfg, teff, logg, mh, abundances, nlte_elems, workdir='.', verbose=False)` Returns `(atmod, nlte_info_file)`.

broaden

`convolve(wave, flux, R=None, lsf=None, lsf_offsets=None, lsf_halfwidth=None, wave_out=None, dw_out=None, dlogw_out=None, rv=0.0, vmac=0.0, vsini=0.0, eps=0.6, pixel_integrate=False)` Returns `(wave_out, flux_out)`.

`lsf_matrix(wave_in, wave_out, R=None, lsf=None, lsf_offsets=None, lsf_halfwidth=None, rv=0.0, nsig=5.0, pixel_integrate=False, nsub=5)` Returns a sparse matrix of shape `(n_out, n_in)`.

`stellar(wave, flux, vmac=0.0, vsini=0.0, eps=0.6)` Stellar broadening on the input grid.

`doppler(wave, rv)`, `output_grid(wave, dw_out=None, dlogw_out=None, wrange=None)`, `rotation_kernel(dlnw, vsini, eps=0.6)`

rv

`ccf_rv(wave, flux, twave, tflux, err=None, vmin=-600.0, vmax=600.0, dv=None, mask=None)` Returns `(rv, rv_err, (v, ccf))`.

`chi2_rv(wave, flux, err, twave, tflux, v0, half=None, nstep=41, mask=None)` Returns `(rv, rv_err, (v, chi2))`.

ferre

`fit(waves, fluxes, errs, rv, grid, ids=None, init=None, workdir='ferre_run', ferre_exe='ferre.x', nthreads=1, extra=None, name='ferre')` Returns `{id: result}`.

`to_star(result, rv=0.0, vmicro=None)` Returns `(star, star_err)`.

Lower-level functions:

- `read_header(path)`
- `grid_wave(hdr)`
- `to_grid_pixels(wave, flux, err, rv, hdr, bad_err=1e10)`
- `write_inputs(workdir, ids, flux, err, hdr, init=None, nthreads=1, name='ferre', extra=None)`
- `read_output(workdir, ids, hdr, name='ferre')`

linefit

`LineFitter(linelists, inst, nodes=None, atmod_lookup=None, atmos_type='marcs', nlte=None, nlte_elems=(), offsets=(-0.6, -0.4, -0.2, 0.0, 0.2, 0.4, 0.6), dw=0.02, n_jobs=1, drv_max=5.0, vmac=3.0, vsini=0.0, cont_deg=1, max_extend=2, workdir=None, verbose=True)`

- `.automask(elements, star, wave_obs, dx=0.3, min_response=0.02, max_blend=0.3, cut=0.25, exclude=None, wrange=None)` returns a list of window dictionaries.
- `.fit(wave, flux, err, star, elements=None, mask=None, star_err=None, vmac='global', iterate=1, sys_vmicro=False, sys_nlte=False, min_pix=4, profile=True, mc=0, seed=None)` returns a `LineFitResult`.
- `.mode(elem)` returns `'CNO-LTE'` or `'NLTE'`.

`LineFitResult`

- Attributes: `.lines`, `.summary`, `.vmac`.
- Methods: `.table()` (astropy Table) and `.write(path)`.

`read_mask(path, element=None)`, `write_mask(mask, path)`

`plot_lines(res, outfile=None, ncols=4, element=None, colors=('k', 'r'))`

`bracket(star, nodes)`, `corner_weights(br, point)` The node-cell geometry used by CNO-LTE mode.

pipeline

`analyse(wave, flux, err, grid, lf, elements, star_id='star', ferre_exe='ferre.x', workdir='turbofit_run', template_params=None, mask=None, nthreads=1, ferre_extra=None, vmicro=None, rv_range=(-600, 600), **fit_kw)` Returns a dict with keys `rv, rv_err, ferre, star, star_err, mask, lines`.

turbogrid (new or changed arguments only)

`write_grid_table(..., nlte=None, nlte_elems=None, broaden_kw=None)`

`nlte_atmod_lookup(teff, logg, mh, am=0.0, cm=0.0)` Returns `'nlte'`.

`write_ferre_grid(...)` Writes `LOGW = 1` automatically for log₁₀-uniform wavelength grids.

For the rest of turbogrid (`make_atmod_lookup`, `make_grid_params`, `run_one`, `build_grid_local`, `write_slurm_array`, `write_parallel_joblist`, `collect_grid`, `convolve_grid_to_R`, `fill_grid_holes`), see `turbogrid_user_guide.pdf`.